

学号： 246353

# 东南大学专业学位型研究生 学位论文开题报告及论文实施计划

院（系、所） 软件学院

学 位 类 别 电子信息硕士

专 业 领 域 软件工程

研究生姓名 李雅扉

指导教师（校内） 沈典

指导教师（校外） 宋明哲

开题报告日期 2026 年 4 月

东南大学研究生院制表

## 填 表 须 知

1、论文开题报告由研究生本人向审议小组报告并听取意见后，由研究生本人填写此表。

2、论文开题报告填写完成后，必须经导师审批，通过后方能提交。

3、博士生应在第四学期内、硕士生应在第三学期内完成此开题报告。开题报告经研究生秘书在网上审核确认（硕士生至少半年、博士生至少一年）后方可申请答辩。

4、研究生开题前应填写查新报告。查新报告对专业学位博士作为必要环节。博士生查新工作可委托图书馆负责，也可在完成网络文献检索类研究生课程的学习或参加学校组织的网络文献检索培训后，自行组织查新检索，自行组织查新需要详细文献查新述评作为附件。自行查新报告须经导师审查后由开题报告审核专家组审核签字（或盖章）。硕士生开题查新参考上述办法，不作硬性要求。

5、本表一式两份，一份研究生自留放入本人“研究生档案材料袋”；一份由院（系、所）保存并归入院（系、所）研究生教学档案。

6、学位类别为：工程硕士；公共管理硕士；法律硕士（非法学）；工商管理硕士；建筑学硕士；风景园林硕士；临床医学硕士；公共卫生硕士；旅游管理硕士；会计硕士；国际商务硕士；资产评估硕士；工程管理硕士；艺术硕士；工程博士；医学博士等。

7、本表下载区：<http://seugs.seu.edu.cn/3676/list.htm>。本表电子文档打印时用A4纸张，格式不变，内容较多可以加页。

# 一、学位论文开题报告

|         |                                  |          |                  |                |     |      |      |     |    |
|---------|----------------------------------|----------|------------------|----------------|-----|------|------|-----|----|
| 论 文 题 目 | 边缘场景下容器化分布式推理任务的细粒度资源感知调度机制研究与实现 |          |                  |                |     |      |      |     |    |
| 研 究 方 向 | 边缘计算                             |          |                  |                |     |      |      |     |    |
| 题 目 来 源 | 国家                               | 部委       | 省                | 市              | 厂、矿 | 自选   | 有无合同 | 经费数 | 备注 |
|         |                                  |          |                  |                |     | √    |      |     |    |
| 题 目 类 型 | 理论<br>研究                         | 应用<br>研究 | 工<br>程<br>技<br>术 | 跨学<br>科<br>研 究 | 其他  | 应用研究 |      |     |    |

## 开题报告内容

### 一、选题依据与工程应用价值

随着大模型推理服务从云数据中心不断向边缘环境延伸，边缘侧正在成为人工智能应用落地的重要运行场景<sup>[1]</sup>。与云端集中式部署不同，边缘环境更贴近工业制造、智能装备、轨道交通等实际业务现场，对实时感知、本地决策和低时延响应具有更强需求。近年来，以大语言模型（Large Language Models, LLMs）为代表的大规模深度学习模型在自然语言处理、代码生成和智能问答等任务中展现出优异性能，并逐步被应用于边缘侧智能交互、设备运维和辅助决策等场景<sup>[2]</sup>。然而，边缘环境中的计算、存储和网络资源通常相对有限<sup>[3]</sup>，传统单机推理模式难以同时满足大模型推理服务在吞吐能力和响应时延方面不断增长的需求。在此背景下，基于多 GPU 协同计算的分布式推理架构，逐渐成为边缘环境下大模型在线推理服务的重要部署方式，并为边缘侧高性能智能服务提供关键计算支撑，如图 1 所示。

在边缘环境中实现高效的大模型分布式推理并非简单的资源扩展问题，任务调度与资源分配贯穿推理服务部署、执行与优化的全过程，其合理性直接影响系统吞吐能力、响应时延和 GPU 资源利用率。对于边缘侧分布式推理系统而言，高效调度需要综合考虑多个相互关联的环节，包括识别不同推理阶段的计算特征与时延需求，分析不同并行策略下任务之间的通信依赖关系，结合集群节点与网络拓扑完成任务部署，以及在资源受限条件下根据任务对 GPU 计算单元、显存容量和显存带宽等资源的需求进行合理分配。上述环节相互耦合，使边缘环境下的大模型分布式推理调度成为一个涉及通信、计算与资源协同优化的复杂系统问题。

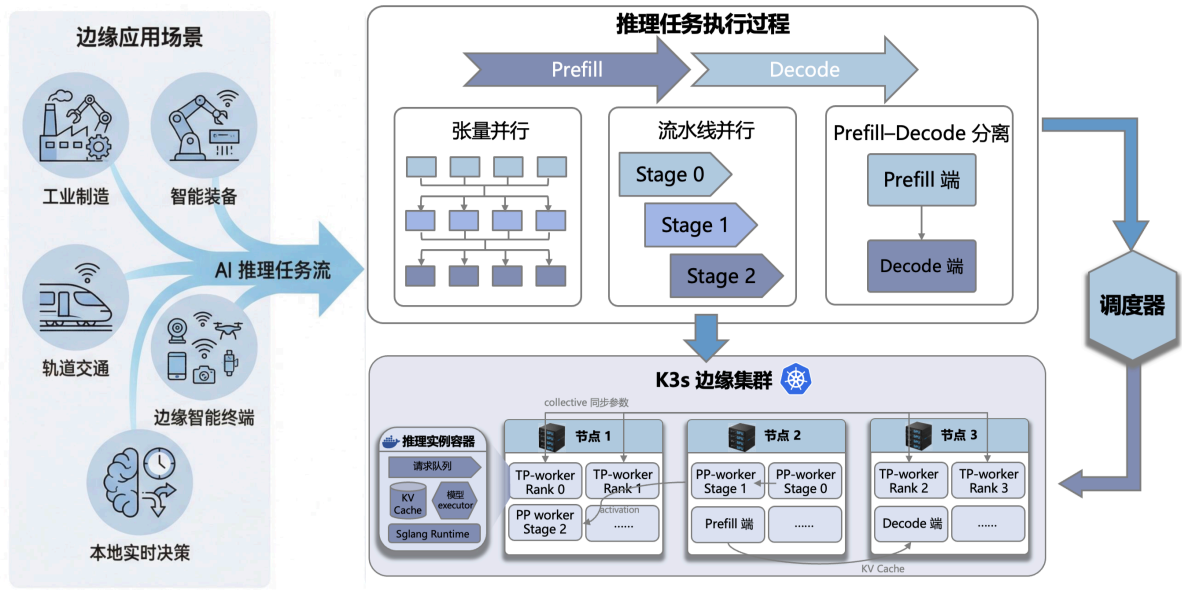


图 1 边缘场景下的分布式推理架构

随着边缘侧大模型推理服务规模不断扩大，以及容器化集群部署和多任务共驻模式的广泛采用<sup>[4][5]</sup>，基于通用资源指标或静态规则的调度方式已难以满足性能优化需求。这一困难主要源于边缘环境下分布式推理任务呈现出的三类典型复杂性。1) **通信异质性**。大模型推理通常包括 Prefill 和 Decode 两个阶段，前者计算量大、并行度高，后者对生成时延更为敏感；同时，Tensor Parallel、Pipeline Parallel 以及 Prefill-Decode 分离等并行方式在模型划分和执行组织上存在明显差异，使任务在运行过程中表现出不同的通信规模、通信频率和通信拓扑特征，从而显著增加调度决策难度。2) **资源竞争性**。为提高有限硬件条件下的 GPU 利用率，多个推理任务常以共驻方式共享同一 GPU 设备<sup>[6][7]</sup>，但不同任务对 SM、显存容量和显存带宽等资源的需求并不一致，不合理的资源分配容易引发计算干扰和性能退化，影响系统整体稳定性。3) **部署受限性**。与云端数据中心相比，边缘环境在计算能力、网络带宽和节点规模等方面更为受限，且边缘侧推理服务通常依托 k3s、Kubernetes<sup>[8]</sup>等容器平台进行集群化部署，任务调度不仅受制于底层资源条件，还受到节点拓扑和网络连接质量的影响<sup>[9]</sup>。因此，这三类特征及其相互作用共同构成了边缘环境下大模型分布式推理调度的核心难点，使系统难以仅依靠传统调度机制同时兼顾通信效率与资源利用率。

针对大模型推理性能优化，现有研究已提出多种推理系统框架，为大模型服务的高效部署提供了重要技术基础。以 vLLM<sup>[10]</sup>、SGLang<sup>[17]</sup>为代表的推理系统在推理执行优化方面展现出较强能力。其中，vLLM 通过 PagedAttention 等机制优化 KV Cache 的显存管理，在保持推理性能的同时提升显存利用效率，从而支撑更高并发的推理请求；SGLang 则通过结构化提示与执行流程优化，提高了复杂推理任务的执行效率与系统支撑能力。总体而言，

这类系统主要通过改进 KV Cache 管理、动态批处理以及推理执行调度机制，在提升系统吞吐量和降低推理延迟方面取得了显著进展。然而，现有方法的优化重点仍主要集中在推理执行层面的性能提升，对于跨节点任务调度以及通信行为优化的研究相对有限。特别是在容器化集群环境中，推理任务的资源分配通常依赖 Kubernetes 调度器，其决策主要依据 CPU、GPU 数量以及节点资源利用率等指标，缺乏对推理任务通信关系的感知能力，也难以支持 GPU 内部计算资源的细粒度调度。

因此，面向边缘场景的复杂分布式推理任务调度仍然面临诸多挑战。由于任务特征与资源约束之间存在复杂耦合关系，现有方法在任务建模与调度优化方面仍存在明显局限，相关问题仍有待进一步深入研究。

### **(1) 分布式推理任务通信模式感知预测难**

分布式大模型推理任务在推理阶段、并行方式以及通信机制等方面存在显著差异，不同任务在通信粒度、通信频率及通信拓扑结构上呈现出复杂且动态变化的特征。例如，Prefill 阶段以大规模通信为主，而 Decode 阶段则表现为高频小规模通信，不同并行策略下通信模式也存在明显差异。这种多维度差异使得推理任务的通信行为在调度前难以被准确感知与刻画。现有调度机制主要依赖 CPU、GPU 数量及节点资源利用率等粗粒度指标进行决策，缺乏对任务通信模式的建模能力，难以在调度前预测不同部署方案下的通信开销，由此导致任务部署后产生大量高开销通信，增加通信延迟并降低整体推理性能。因此，如何在调度前对分布式推理任务的通信模式进行有效建模与预测，构建通信感知的调度决策依据，成为亟需解决的关键问题。

### **(2) GPU 共享环境下资源竞争协调难**

在边缘计算环境中，由于 GPU 资源相对受限，多任务共驻运行成为提升设备利用率的重要方式，不同推理任务往往需要共享 Streaming Multiprocessor (SM)、显存容量和显存带宽等关键硬件资源。然而，不同任务在计算强度、显存占用、带宽敏感性和执行时长等方面存在显著差异，多个任务共驻运行容易在 GPU 内部形成复杂的资源竞争与性能干扰。一方面，粗粒度的独占式分配会导致 GPU 利用率不足；另一方面，仅采用静态比例划分的 GPU 共享方式而不考虑任务在计算强度、显存占用和带宽需求上的差异，则容易引发资源争用，导致任务时延上升、吞吐下降，难以保证推理性能。现有调度机制大多仍以整卡级资源分配为主，缺乏面向 GPU 内部计算资源的细粒度管理能力，难以根据任务特征动态协调 SM、显存和带宽等资源的使用关系。因此，如何在 GPU 共享环境下刻画任务间的资源

竞争特征，设计细粒度资源调度机制，并实现资源利用率与推理性能之间的有效平衡，仍然是分布式推理系统中的关键研究问题。

综上所述，在分布式大模型推理系统中，复杂的通信行为与 GPU 资源共享机制共同增加了推理任务调度的难度。现有推理框架与容器调度机制在通信感知能力和细粒度资源管理方面仍存在明显不足，难以满足复杂推理任务场景下的性能优化需求。因此，有必要围绕分布式推理任务通信模式抽取、通信代价预测与 GPU 资源调度机制开展深入研究。基于此，本课题面向边缘场景下容器化分布式推理任务，拟研究一种细粒度资源感知调度机制，通过构建推理任务通信模式抽取与代价预测方法，并结合多阶段资源调度策略，实现推理任务部署与 GPU 资源分配的协同优化，从而提升分布式推理系统的整体性能和资源利用效率。

## 二、国内外研究现状与发展动态

### 2.1 大模型推理系统研究现状

随着大语言模型（LLM）的快速发展，尤其是模型参数规模从百亿级别扩展到万亿级别，推理过程中的计算、内存、通信和延迟等方面的挑战，成为影响大模型实际应用的关键因素。研究者们将大量精力投入到推理部署阶段的系统优化，提出了多种优化策略以提升推理性能和效率。

在推理执行优化方面，研究者围绕内存管理与计算效率开展了大量工作。PagedAttention<sup>[10]</sup>及其实现系统 vLLM 通过引入类似虚拟内存分页的机制，对 KV Cache 进行动态管理，有效缓解了显存碎片化问题，并显著提升了系统吞吐能力。在解码过程优化方面，Speculative Decoding<sup>[11]</sup>通过“小模型生成、大模型验证”的方式减少自回归解码的串行开销，实现了推理延迟的有效降低；Medusa<sup>[12]</sup>进一步通过多解码头机制实现多 token 并行预测，在不增加额外模型的情况下提升了生成效率。此外，AWQ<sup>[13]</sup>等量化方法通过降低模型权重精度减少显存占用和访存开销，PowerInfer<sup>[14]</sup>则利用激活稀疏性优化推理计算路径，使得大模型能够在资源受限设备上高效运行。在底层算子优化方面，FlashAttention-2<sup>[15]</sup>等方法通过优化注意力计算的数据访问模式，提高了 GPU 计算与存储效率，从而进一步降低推理延迟。总体来看，这些方法从不同层面提升了单节点推理系统的执行效率，为大规模推理服务提供了基础支撑。

在系统框架层面，面向在线推理服务的系统设计也不断演进。Orca<sup>[16]</sup>通过动态批处理机制提高了 GPU 利用率，缓解了请求长度不一致带来的资源浪费问题；vLLM<sup>[10]</sup>在此基础上引入持续批处理与高效内存管理机制，实现了高吞吐推理服务；SGLang<sup>[17]</sup>则通过结构化执行模型与共享前缀优化机制，提升了复杂推理任务的执行效率。此外，FlexGen<sup>[18]</sup>通过将模

型权重与 KV Cache 在 GPU、CPU 和外部存储之间进行分层管理，实现了在资源受限环境下运行超大模型的能力。这些系统从资源管理、请求调度和执行优化等方面构建了较为完善的大模型推理服务框架。

在分布式推理机制方面，现有研究主要围绕模型并行与系统扩展展开。张量并行（Tensor Parallelism）通过对模型内部计算进行切分，实现多 GPU 协同计算，但需要频繁的集合通信操作；流水线并行（Pipeline Parallelism）通过层间划分降低单设备存储压力，但会引入激活传输与流水线气泡问题；混行并行策略则综合利用不同并行方式，以在计算效率与通信开销之间取得平衡<sup>[19][20]</sup>。随着并行策略的复杂化，推理任务在执行过程中呈现出多样化的通信行为模式，不同并行方式下的中间结果同步、激活传输及缓存共享均对系统性能产生重要影响。因此，通信开销已从传统的系统实现问题演变为分布式推理系统设计中的核心瓶颈<sup>[20][21]</sup>。

综上所述，现有研究在推理执行优化、系统框架设计以及分布式并行机制等方面取得了显著进展，有效提升了大模型推理系统的性能与可扩展性。然而，这些工作主要集中在推理计算与系统执行效率的提升，对于分布式推理过程中通信行为的建模与分析关注仍相对不足，缺乏面向通信特征的任务调度与资源分配机制。这一问题在多节点部署与复杂并行策略并存的环境中尤为突出，也为后续容器集群资源调度研究提供了重要的研究动机。

## 2.2 容器集群资源调度研究现状

近年来，随着深度学习与大模型推理任务的快速发展，容器集群调度问题从传统的 CPU 与内存分配，逐步扩展到涵盖 GPU、显存、网络带宽等多维异构资源的综合调度问题<sup>[22]</sup>，其复杂性显著提升<sup>[23][24]</sup>。

在基础调度机制方面，早期集群资源管理系统（如 Mesos<sup>[23]</sup>）采用双层调度架构，通过资源邀约机制实现资源分配与任务调度的解耦。该类方法具有良好的灵活性，但难以实现全局最优调度。随后，以 Kubernetes 为代表的容器编排平台逐渐成为主流，其调度器通过“过滤—打分”机制，根据资源需求与节点状态选择合适的运行节点<sup>[25][26]</sup>。这类调度方式能够满足通用应用场景，但其主要依赖静态资源指标，缺乏对任务执行特征和动态负载的感知能力，在复杂应用场景中存在一定局限性<sup>[27]</sup>。

在多资源调度方面，主导资源公平（Dominant Resource Fairness, DRF）为多维资源分配提供了重要理论基础，通过平衡不同用户在其主导资源上的占比，实现多资源环境下的公平分配<sup>[28]</sup>。然而，DRF 在实际应用中容易导致资源碎片化问题，并且难以考虑任务之间的拓扑关系与放置约束。为进一步提升资源利用效率，研究者提出了基于资源打包的调度策略，将任务尽可能紧凑地部署到节点上，以减少资源浪费。典型工作如 Tetris<sup>[29]</sup>通过在公平性与资

源利用率之间进行权衡,实现了多资源场景下的高效调度。Struhár<sup>[35]</sup>针对边缘环境中实时容器因共享资源干扰而难以保证时间可预测性的问题,提出了离线初始部署与在线运行时重调节的分层资源编排框架,通过 CPU 预算控制、资源再分配和必要时容器迁移来维持实时性能。总体来看,公平性与资源利用率之间的权衡成为容器集群调度中的核心问题。

随着 AI/ML 任务的广泛应用, GPU 资源调度逐渐成为研究重点<sup>[34]</sup>。Tiresias<sup>[30]</sup>通过引入基于 Gittins 指数的调度策略,在无需预知任务执行时间的情况下,有效降低了作业完成时间(JCT),并缓解了队头阻塞问题。Gandiva<sup>[31]</sup>进一步通过时间片共享与任务抢占机制,实现 GPU 资源的细粒度复用,提高了资源利用率与调度灵活性。与之不同, Synergy<sup>[24]</sup>从任务特性出发,通过分析不同任务在计算、内存和网络等资源上的瓶颈差异,实现资源协同调度,从而减少任务间干扰并提升整体系统吞吐。Ziyi<sup>[32]</sup>针对多 GPU 推理中干扰建模、动态任务调度和资源碎片问题,提出 InSS 智能调度协调器,通过建立干扰感知延迟分析模型和两阶段智能调度器,在满足 SLO 的同时提升吞吐量。Zhenqian<sup>[33]</sup>关注 GPU 集群中训练任务资源紧张而推理集群又存在周期性空闲、导致整体利用率低且训练 JCT 偏高的问题,提出通过动态扩缩容调度和快速弹性训练恢复机制,让训练任务临时借用推理侧空闲 GPU 并在推理负载回升时低开销归还资源,实现保障推理 SLO 并降低平均训练作业完成时间。这些研究表明,面向 AI 负载的调度方法逐渐从传统资源分配转向任务特征感知与性能优化。

在资源粒度与硬件支持方面, GPU 共享与虚拟化技术为细粒度调度提供了新的手段。以 NVIDIA MIG 为代表的硬件机制能够将单个 GPU 划分为多个独立实例,从而支持多任务并发执行,提高设备利用率<sup>[36]</sup>。然而,这类方法仍需要调度系统进行有效协调,例如在不同任务之间合理分配 GPU 实例,或动态调整资源划分策略,以适应负载变化。此外,显存管理与共享也成为重要研究方向,如何在保证任务性能的前提下实现显存复用与动态分配,是当前亟待解决的问题。

综上所述,容器集群资源调度研究已经从早期的粗粒度资源分配,发展到面向多维异构资源的精细化调度,并在公平性、效率与性能优化之间不断权衡。然而,现有方法主要基于资源容量与负载信息进行决策,缺乏对分布式推理任务通信行为与执行特征的联合建模,难以适应大模型推理场景下复杂的通信模式与资源需求。因此,如何在容器集群环境中实现通信感知与资源感知的协同调度,成为当前亟需进一步研究的重要方向。

## 2.3 研究现状总结

综合现有研究进展可以看出,大模型推理系统与容器集群资源调度在推理执行优化与资源管理方面均取得了重要进展,为大规模智能应用的落地提供了坚实基础。在推理系统层面,



围绕 KV Cache 管理、解码算法优化、模型压缩及高效算子设计等关键问题，相关研究显著提升了单节点推理性能；同时，vLLM、SGLang 等系统框架的出现推动了推理服务向工程化与系统化方向发展。在分布式推理场景中，张量并行、流水线并行及混合并行机制有效支撑了大模型的规模扩展，但也引入了复杂的通信行为，使通信开销逐渐成为影响系统性能的重要因素。在容器集群调度方面，Kubernetes 等平台提供了统一的资源管理基础，DRF、Tetris 等方法在多资源公平性与利用率之间实现了初步平衡，Tiresias、Gandiva 及 Synergy 等面向 AI 负载的调度策略进一步提升了 GPU 资源利用效率。然而，现有研究在系统层与调度层之间仍存在明显割裂：一方面，推理系统优化主要关注计算效率与内存管理，缺乏对任务通信特征的深入建模；另一方面，容器调度方法多依赖静态资源指标，难以刻画分布式推理任务在通信模式与计算行为上的差异。在多节点部署与复杂并行策略并存的环境下，这种割裂导致调度策略难以兼顾通信开销与资源竞争，限制了系统整体性能的进一步提升。因此，如何在容器集群环境中引入通信感知与资源感知机制，实现分布式推理任务的协同调度与细粒度资源分配，成为当前亟需深入研究的关键问题。

### 三、研究目标与研究内容

#### 3.1 研究目标

本研究围绕分布式大模型推理任务在容器集群环境中的高效部署与运行需求，针对推理任务通信行为复杂以及 GPU 资源共享条件下调度效率不足等问题，开展通信感知与资源协同优化的推理任务调度方法研究。针对分布式推理任务通信模式复杂且难以刻画的问题，研究如何对推理任务的通信行为进行建模与分析，构建统一的通信模式描述方法，为调度决策提供依据。针对容器集群环境中 GPU 资源分配粒度粗、任务共驻导致资源竞争严重的问题，研究如何设计细粒度资源调度机制，实现多任务环境下资源利用率与推理性能的平衡。最终，构建面向边缘场景的分布式推理调度框架，结合通信模式抽取与代价预测方法以及通信感知与细粒度资源协同调度机制，实现推理任务部署与资源分配的协同优化，在验证方法有效性的基础上，为大模型推理系统提供高效、可扩展的调度解决方案。

#### 3.2 研究内容

为实现上述研究目标，针对分布式大模型推理系统中通信行为复杂及 GPU 资源共享条件下调度效率不足的问题，本研究围绕通信模式建模与资源调度协同优化展开。首先，针对分布式推理任务通信模式感知预测难的问题，研究面向分布式推理任务的通信模式抽取与代价预测方法，通过对通信规模、频率及拓扑特征进行建模，实现通信行为的结构化表达，为调度决策提供依据。其次，针对 GPU 共享环境下资源竞争协调难的问题，研究基于通信特征的任务部署优化方法与细粒度 GPU 资源调度机制，构建集群级部署优化与节点级

资源分配的通信感知与细粒度资源协同调度机制，实现通信开销与计算资源利用的协同优化。研究拟通过系统设计与实现，在边缘场景 k3s 容器集群环境中构建分布式推理调度系统，对所提出方法进行集成验证，从通信开销、推理延迟及资源利用率等方面评估其有效性。以上研究内容的关系如图 2 所示：

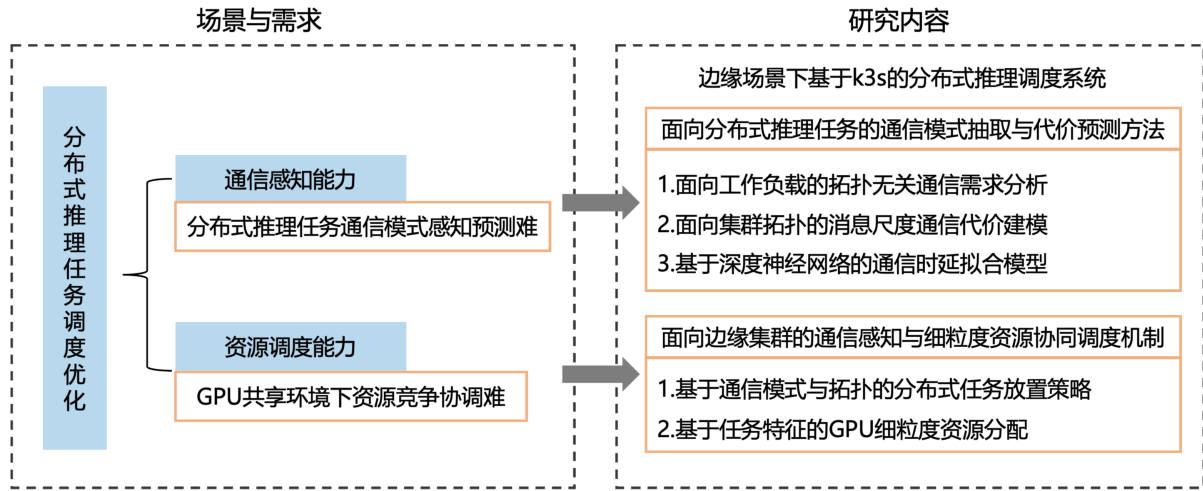


图 2 研究内容框架

### 3.2.1 面向分布式推理任务的通信模式抽取与代价预测方法

针对分布式大模型推理任务在不同并行策略与执行阶段下通信行为复杂且难以建模的问题，本研究拟开展推理任务通信模式抽取与代价预测方法，通过通信特征建模与模式抽取实现任务级通信行为刻画。在通信行为建模方面，分析大模型推理过程中 Prefill 与 Decode 阶段在计算模式与通信需求上的差异，结合 Tensor Parallel (TP)、Pipeline Parallel (PP) 等并行策略，提取通信规模、通信频率及通信拓扑等关键特征，构建统一的通信特征表示方法。在此基础上，基于推理框架（如 SGLang）对分布式推理执行过程进行分析，构建任务间通信关系模型，实现通信行为的结构化表达。在通信模式抽取与代价预测方面，设计面向推理任务的通信模式分类方法，根据通信特征对任务进行模式划分，并构建通信矩阵或通信图结构，刻画分布式任务切片间的通信依赖关系。通过通信模式抽取与代价预测，将复杂的分布式通信行为转化为可用于调度决策的结构化信息，为后续任务部署优化提供支撑。

### 3.2.2 面向边缘集群的通信感知与细粒度 GPU 资源协同调度机制

针对容器集群环境中调度策略缺乏通信感知能力以及 GPU 资源分配粒度不足的问题，本研究拟设计通信感知与细粒度资源协同调度机制，实现任务部署与资源分配的协同优化。在第一阶段（集群级任务部署）中，结合通信代价预测结果与集群网络拓扑信息，构建通信代价模型，根据任务间通信强度优化任务在不同节点间的放置位置，减少跨节点通信开销，提高系统整体通信效率。在第二阶段（节点级资源调度）中，针对多任务共驻场景下 GPU 资

源竞争问题，设计细粒度 GPU 资源分配方法，通过动态调整 Streaming Multiprocessor (SM) 等计算资源分配比例，实现任务间资源隔离与协同运行，降低资源竞争带来的性能干扰。通过通信感知部署和细粒度资源调度的两阶段机制，实现通信优化与计算资源优化的统一，从而提升分布式推理系统的整体性能。

### 3.2.3 边缘场景下基于 k3s 的分布式推理调度系统

为验证上述方法的有效性，本研究拟构建基于 k3s 轻量级容器集群的分布式推理调度系统，实现通信模式抽取与代价预测方法和通信感知与细粒度资源协同调度机制的集成。在系统设计方面，构建推理任务输入模块，实现推理任务特征提取，形成统一的调度特征表示；构建调度决策模块，实现基于通信模式的任务部署策略与 GPU 资源分配策略；构建执行与监控模块，实现任务运行状态监测与性能评估。在系统实现方面，基于 SGLang 推理框架与 k3s 集群环境部署分布式推理服务，实现调度策略的动态执行与资源管理，并在多任务共驻与分布式推理场景下开展实验验证。通过系统实现与实验分析，从通信开销、推理延迟及资源利用率等指标评估所提方法的有效性，为分布式大模型推理系统的调度优化提供实践支撑。

## 四、实施方案及可行性分析

### 4.1 面向分布式推理任务的通信模式抽取与代价预测方法

本研究拟面向边缘环境下分布式大模型推理场景中的通信感知调度问题，构建一种从工作负载规模到通信代价的结构化建模方法，用于支撑不同部署方案的代价比较与资源选择。在给定模型配置、切分方式与候选部署拓扑的条件下，该方法以通信行为分析为基础，围绕需求建模—代价建模—调度输入生成展开，形成面向调度决策的通信建模闭环。其核心思路是将通信建模划分为两个相互衔接的阶段：第一阶段抽取与拓扑无关的通信需求画像，描述推理任务在不同工作负载下的通信规模、通信频率与通信结构；第二阶段建模与拓扑相关的通信代价，建立通信需求与链路开销之间的映射关系。通过两阶段结果的组合，实现对通信时间的结构化预估，并将其作为调度器进行部署优化和资源选择的重要依据，如图 3 所示。

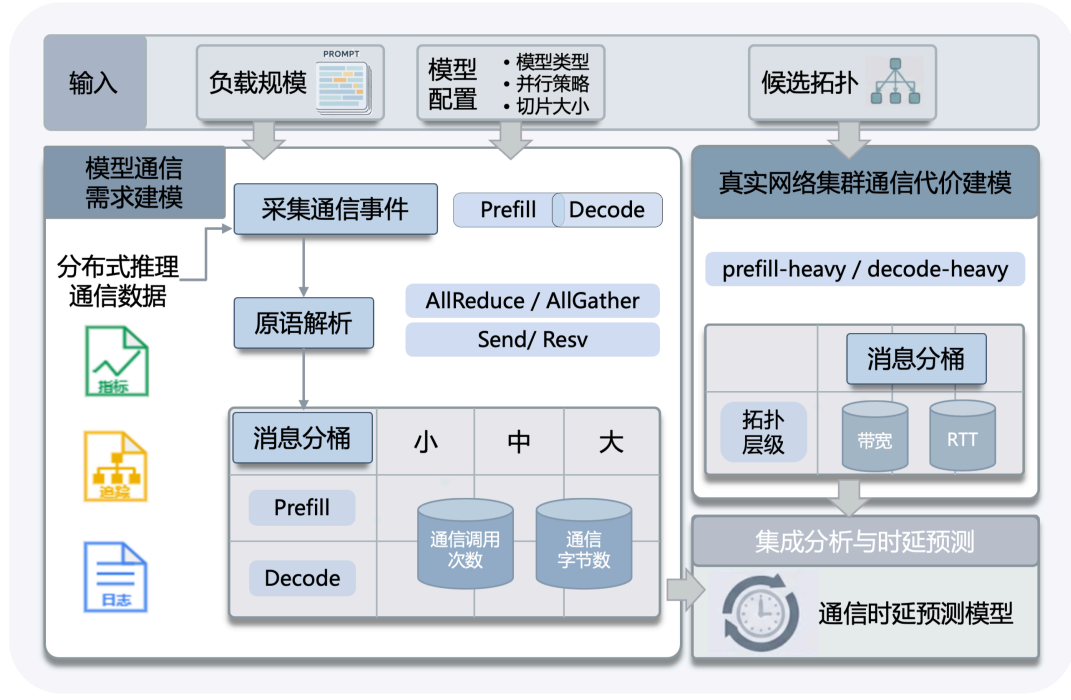


图 3 面向分布式推理任务的通信模式抽取与代价预测方法

推理工作负载表示为：

$$w = (L, M) \quad (1)$$

其中， $L$  表示输入 prompt 长度， $M$  表示输出 token 上限。设模型与并行配置表示为：

$$c = (\text{model}, \text{parallel\_form}, \text{parallel\_size}) \quad (2)$$

其中， $\text{parallel\_form}$  对应 TP、PP、PD 分离并行形式， $\text{parallel\_size}$  表示并行规模。设候选部署拓扑为  $\pi$ 。本研究的目标是构造通信代价估计函数  $T_{\text{comm}}(w, c, \pi)$ ，反映工作负载规模变化所导致的通信模式变化，以及这些变化在不同部署拓扑上的代价差异。

#### 4.1.1 面向工作负载的拓扑无关通信需求分析

第一阶段关注的问题是在不引入具体网络拓扑代价的前提下，给定工作负载在特定模型配置与并行方式下会产生怎样的通信需求。为此，本阶段将推理过程划分为 Prefill 与 Decode 两个阶段，并在每个阶段内部进一步按消息尺度进行分桶，以保留通信规模结构对性能的影响。设消息桶集合为：

$$U = \{U_1, U_2, U_3\} \quad (3)$$

其中  $U_1$ 、 $U_2$ 、 $U_3$  分别表示小消息、中等消息和大消息三个消息尺度区间。定义桶划分函数：

$$g(m) = \begin{cases} U_1, & m < T_1 \\ U_2, & T_1 < m < T_2 \\ U_3, & m > T_2 \end{cases} \quad (4)$$

其中,  $m$  表示单次通信事件的逻辑消息大小。采用分桶机制的目的在于进一步保留不同消息尺度下的结构性差异, 避免仅以通信总量描述工作负载。

事件级别上, 对每条通信事件定义为:

$$e = (\phi_e, o_e, m_e, p) \quad (5)$$

其中,  $\phi_e$  表示事件所属推理阶段,  $o_e$  表示通信原语类型,  $m_e$  表示该事件的逻辑消息大小,  $p$  表示并行规模。考虑到逻辑消息大小并不直接等价于网络实际负担, 本研究进一步引入两类事件级折算量: 等效字节需求  $b_e^{eq}$  与等效轮次需求  $r_e^{eq}$ , 并将其建模为通信原语类型与并行规模的函数:

$$b_e^{eq} = \alpha_{o_e}(p) \cdot m_e \quad (6)$$

$$r_e^{eq} = \beta_{o_e}(p) \quad (7)$$

采用 ring-style 近似对典型 collective 原语进行折算, 使得不同原语带来的网络负担能够在统一尺度下进行比较与聚合。对于 AllReduce, 有

$$\alpha_{AR}(p) = \frac{2(p-1)}{p} \quad (8)$$

$$\beta_{AR}(p) = 2(p-1) \quad (9)$$

对于 AllGather, 有

$$\alpha_{AG}(p) = \frac{p-1}{p} \quad (10)$$

$$\beta_{AG}(p) = p-1 \quad (11)$$

在此基础上, 进一步在阶段 $\times$ 消息桶的粒度上对通信事件进行聚合。对于每个阶段  $\phi$  和消息桶  $u$ , 分别定义其等效字节需求与等效轮次需求为:

$$B_{\phi,u}^{eq}(w, c) = \sum_{\{e: \phi_e = \phi, g(m_e) = u\}} b_e^{eq} \quad (12)$$

$$R_{\phi,u}^{eq}(w, c) = \sum_{\{e: \phi_e = \phi, g(m_e) = u\}} r_e^{eq} \quad (13)$$

由此, 第一阶段最终输出一个分阶段、分消息尺度桶的通信需求向量:

$$D(w, c) = \{(B_{\phi,u}^{eq}, R_{\phi,u}^{eq}) \mid \phi \in \{\text{pre}, \text{dec}\}, u \in U\} \quad (14)$$

该通信需求向量作为工作负载在给定模型与并行配置下的拓扑无关通信需求模式, 与仅使用总通信量相比, 不仅建模了通信总量, 还显式保留消息尺度分布及通信原语差异, 因此能够反工作负载规模变化所引发的通信模式迁移。例如, 当输出 token 数增大时

Decode 阶段中小消息高频通信占比可能上升；而当输入 prompt 增长时，Prefill 阶段中大消息通信需求往往更为突出。

#### 4.1.2 面向集群拓扑的消息尺度通信代价建模

第二阶段关注的问题是：对给定候选拓扑  $\pi$ ，不同消息尺度下的通信的有效代价。为使第二阶段与第一阶段保持粒度一致，本阶段同样采用 small、medium、large 三个消息桶进行建模，并在此基础上进一步引入拓扑层级划分，以刻画不同部署关系下的通信代价差异。本阶段将候选部署拓扑划分为三类拓扑层级：

$$\Pi = \{\pi_{local}, \pi_{rack}, \pi_{cross}\} \quad (15)$$

其中， $\pi_{local}$  表示同节点部署， $\pi_{rack}$  表示跨节点同机架部署， $\pi_{cross}$  表示跨机架部署。该划分反映了集群环境中不同任务放置关系所对应的典型通信路径差异。由于同节点、同机架跨节点与跨机架三类拓扑在通信链路层次、路径长度及共享网络资源上存在显著不同，因此对每一类拓扑层级  $\pi \in \Pi$ ，定义其拓扑代价为：

$$C(\pi) = \{(BW_u(\pi), RTT_u(\pi))\}_{u \in U} \quad (16)$$

其中， $BW_u(\pi)$  表示拓扑  $\pi$  在桶  $u$  对应消息尺度下的有效带宽， $RTT_u(\pi)$  表示拓扑  $\pi$  在桶  $u$  对应消息尺度下的等效单轮时延。

按 small、medium、large 三类消息规模分别测量：small 消息桶对应的小消息带宽与 RTT、medium 消息桶对应的中等消息带宽与 RTT、large 消息桶对应的大消息带宽与 RTT，从而得到包含通信库实现、软件栈、链路特性与系统开销在内的网络实际通信代价。

#### 4.1.3 基于深度神经网络的通信时延拟合模型

给定工作负载  $w$ 、配置  $c$  与候选拓扑  $\pi$ ，本研究将各阶段的通信时间建模为：

$$T_\phi(w, c, \pi) = \sum_{u \in U} \left( \frac{B_{\phi,u}^{eq}(w, c)}{BW_u(\pi)} + R_{\phi,u}^{eq}(w, c) \cdot RTT_u(\pi) \right), \quad \phi \in \{\text{pre}, \text{dec}\}. \quad (17)$$

该建模具体含义指，第一阶段给出工作负载在各阶段、各消息尺度桶中的通信需求；第二阶段给出候选拓扑在相应消息尺度下的有效带宽与 RTT；最终总通信代价由所有消息桶的 bytes / BW + rounds × RTT 累加得到。因此，通信时延由工作负载在不同消息尺度下的需求分布与拓扑对这些需求的响应共同决定。

在完成结构化建模后，本研究进一步采用神经网络对通信时延进行拟合。具体地，工作负载特征、配置特征以及拓扑代价特征分别表示为  $F_w$ 、 $F_c$  和  $F_\pi$ 。其中， $F_w$  用于描述输入长度、输出长度等工作负载规模信息， $F_c$  用于描述模型类型、并行方式、并行

规模及切片方式等执行配置,  $F_\pi$  用于刻画候选拓扑层级及其在不同消息尺度区间下的带宽与时延代价特征。

在此基础上, 构造输入表示为:

$$X(w, c, \pi) = (F_w, F_c, F_\pi) \quad (18)$$

构建通信时延拟合模型:

$$\hat{T}_{\text{comm}}(w, c, \pi) = F_\theta(X(w, c, \pi)) \quad (19)$$

其中,  $F_\theta(\cdot)$  表示由深度神经网络参数化的非线性映射函数,  $\hat{T}_{\text{comm}}(w, c, \pi)$  表示模型预测的通信时延。该模型以工作负载规模、执行配置以及拓扑在不同消息尺度区间下的通信代价作为输入, 学习它们与通信时延之间的映射关系, 从而实现对未直接测量组合下通信时延的预测。

模型的优化目标定义为最小化以均方误差 MSE 为基础的损失函数:

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \left( T_{\text{comm}}^{(i)} - \hat{T}_{\text{comm}}^{(i)} \right)^2 \quad (20)$$

其中,  $\hat{T}_{\text{comm}}^{(i)} = F_\theta(X_i)$  表示模型对第  $i$  个样本通信时延的预测值。通过最小化上述损失函数, 可获得最优参数  $\theta^*$ , 以此实现对新“工作负载——配置——拓扑”组合下通信时延的估计, 为后续调度器进行部署优化和资源选择提供通信感知的依据。

#### 4.2 面向边缘集群的通信感知与细粒度资源协同调度机制

本研究拟面向容器集群环境下分布式 AI 推理任务的部署与执行优化问题, 构建一套基于边缘集群的通信感知与细粒度资源协同调度机制, 用于支撑任务在集群级的放置决策与节点级的资源协调。该方案以通信模式抽取与代价预测结果为基础, 围绕“集群级任务部署——节点级资源调度”两层决策展开, 核心思路是: 第一阶段结合任务通信模式与集群拓扑信息, 在全局范围内优化任务放置位置, 以降低跨节点通信开销; 第二阶段在给定部署结果的基础上, 进一步面向节点内部多任务共驻场景, 对 GPU 计算资源进行细粒度协调分配, 以缓解资源竞争带来的性能干扰。通过上述两阶段实施方案, 实现通信优化与计算资源优化的协同, 从而提升容器集群环境下分布式推理任务的整体执行效率。概览图 4 如图所示:

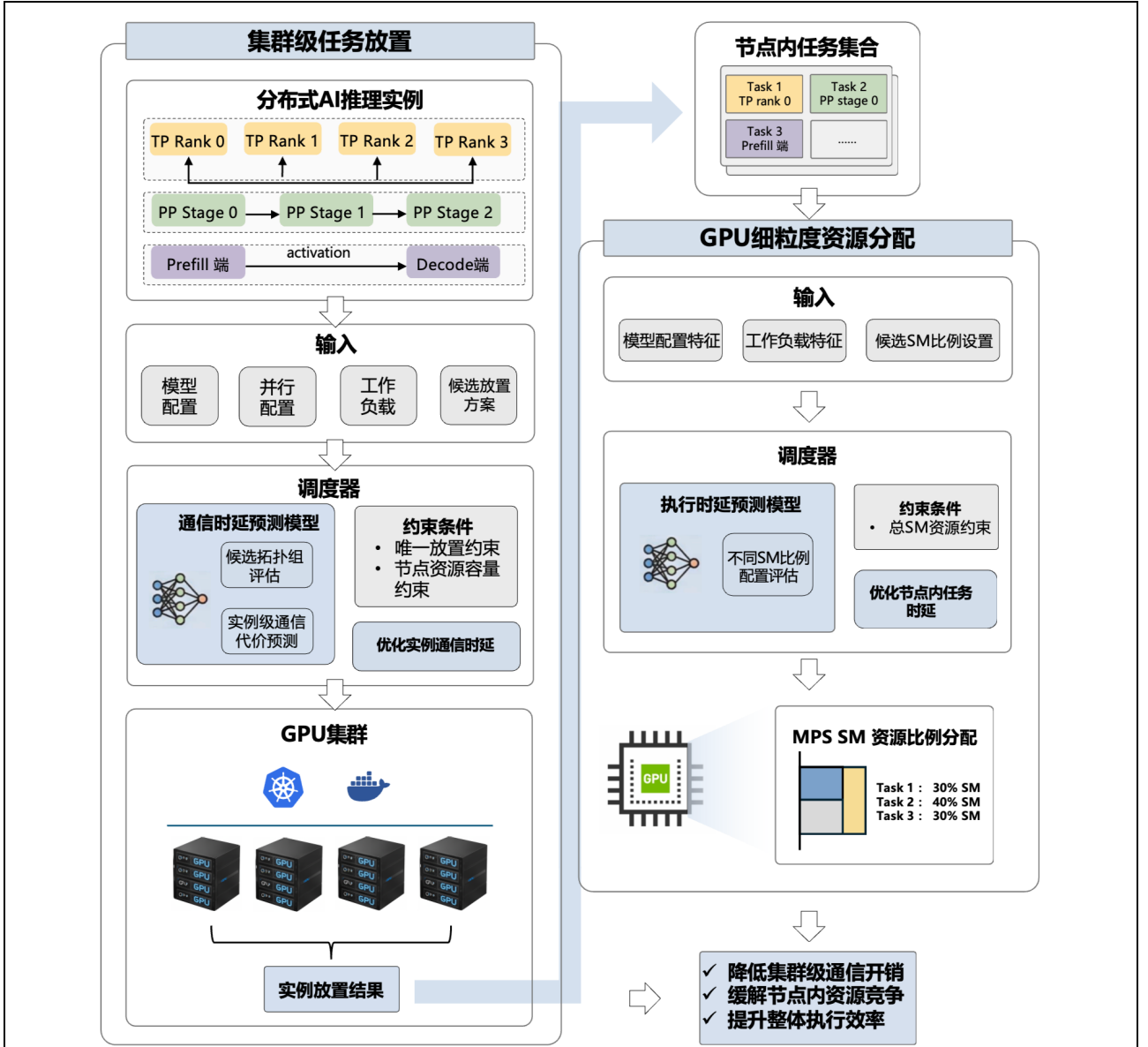


图 4 面向边缘集群的通信感知与细粒度资源协同调度机制

#### 4.2.1 基于通信模式与拓扑的分布式任务放置策略

第一阶段关注的核心问题是：在给定任务工作负载、模型配置以及候选部署拓扑的条件下，如何综合利用通信需求与拓扑代价信息，基于通信时延拟合模型为任务选择更优的跨节点放置位置，从而降低跨节点通信开销并提升整体执行效率。本研究将该过程建模成一个最小化通信代价的任务放置建模问题，并在此基础上设计相应的分布式任务放置策略。

设 GPU 计算节点集合为  $Q = \{1, 2, \dots, q\}$ ，待调度的分布式推理实例集合为  $I = \{I_1, I_2, \dots, I_m\}$ ，其中每个推理实例  $I_r$  在给定并行策略下会被划分为若干并行子任务，记其并行子任务集合为  $V_r = \{v_{r1}, v_{r2}, \dots, v_{rn_r}\}$ 。



并行子任务可以由不同并行机制具体实例化：在张量并行场景下，对应为不同的 TP rank；在流水线并行场景下，对应为不同的 PP stage；在 Prefill—Decode 分离场景下，对应为 Prefill 端与 Decode 端两个异构子任务。所有推理实例的并行子任务联合构成全局并行子任务集合：

$$V = \bigcup_{r=1}^m V_r \quad (21)$$

因此可以得到调度变量：

$$y_{ij} = \begin{cases} 1, & \text{并行子任务 } v_i \text{ 运行在 GPU 计算节点 } j \\ 0, & \text{并行子任务 } v_i \text{ 未运行在 GPU 计算节点 } j \end{cases} \quad (22)$$

对于每个子任务，只能被分配到一个计算节点，不会被拆分至多个计算节点，因此有：

$$\sum_{j \in Q} y_{ij} = 1, \forall v_i \in V \quad (23)$$

在调度变量  $Y = \{y_{ij}\}$  给定的条件下，集合  $V_r$  中各并行子任务会被映射到若干 GPU 计算节点上，由此得到该实例对应的放置关系与拓扑层级特征。记该实例在当前放置方案下的候选拓扑组为  $\pi_r(Y)$ 。将其输入前文训练得到的通信时延拟合模型  $F_\theta(\cdot)$ ，即可得到实例  $I_r$  在当前放置方案下的预测通信时延：

$$\hat{T}_r^{comm} = F_{\theta^*}(X(I_r, \pi_r(Y))) \quad (24)$$

对于每个任务  $v_i$ ，需要的资源占用  $D_i = (\rho_i^{mem}, S_i^{res})$ ， $\rho_i^{mem}$  表示显存需求， $S_i^{res}$  表示资源主导类型，如算力、带宽、显存主导等，不参与硬约束。

因此，计算节点  $j$  的总显存资源占用表示为  $M_{gpu}^{(j)} = \sum_{i \in V} y_{ij} \cdot \rho_i^{mem}$ ，每个计算节点的已使用的显存资源应当小于其最大资源上限  $M_{total}$ ，即有：

$$M_{gpu}^{(j)} \leq M_{total}, \forall j \in Q \quad (25)$$

由此，第一阶段分布式任务放置的优化目标可定义为：在满足唯一放置约束与节点资源约束的前提下，使所有推理实例在当前放置结果下的预测通信时延总代价最小。其目标函数可表示为：

$$\min_Y J_{comm} = \sum_{I_r \in \mathcal{I}} \hat{T}_r^{comm} = \sum_{I_r \in \mathcal{I}} F_\theta(X(I_r, \pi_r(Y))) \quad (26)$$

通过对子任务级调度变量  $y_{ij}$  的优化, 形成实例级候选拓扑组  $\pi_r(Y)$ , 再借助通信时延拟合模型对该拓扑组进行整体代价评估, 最终实现面向实例级通信总代价最小化的任务放置决策。

#### 4.2.2 基于任务特征的 GPU 细粒度资源分配策略

第二阶段关注同一 GPU 节点内多任务共驻场景下的细粒度资源分配问题, 为了在 GPU 节点中共享 GPU 算力, 本研究采用了英伟达 MPS 技术。为得到不同推理模型在不同 MPS/SM 资源配置下的执行性能, 本研究首先构建节点级任务执行时延拟合模型。

对于部署在同一 GPU 节点上的任务  $a_i$ , 记其模型类型特征、工作负载特征以及 MPS SM 资源比例特征分别为  $F_{m,i}$ 、 $F_{w,i}$  和  $F_{s,i}$ 。其中,  $F_{m,i}$  用于描述任务所属模型类型,  $F_{w,i}$  用于描述输入长度、输出长度等工作负载规模信息,  $F_{s,i}$  用于描述该任务在当前节点内被分配到的 MPS SM 资源比例设置。由此构造任务  $a_i$  的输入表示为:

$$X_i = (F_{m,i}, F_{w,i}, F_{s,i}) \quad (27)$$

设对应的真实执行时延为  $T_i^{exe}$ , 则可建立节点级任务执行时延拟合模型:

$$\hat{T}_i^{exe} = G_\theta(X_i) \quad (28)$$

其中,  $G_\theta(\cdot)$  表示由深度神经网络参数化的非线性映射函数,  $\hat{T}_i$  表示模型预测得到的任务执行时延。该模型以任务类型、工作负载以及 SM 资源分配比例作为输入, 用于学习三者与任务执行时延之间的映射关系, 从而预测在不同细粒度资源配置下的任务性能表现。

模型的优化目标定义为最小化以均方误差 MSE 为基础的损失函数:

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N (T_i^{exe} - \hat{T}_i^{exe})^2 \quad (29)$$

通过训练得到最优参数  $\theta^*$  后, 即可利用模型  $G_{\theta^*}(\cdot)$  对不同任务在不同 MPS/SM 资源比例设置下的执行时延进行预测。

设节点  $q_j$  上的任务集合为  $A_j = \{a_1, a_2, \dots, a_{n_j}\}$ , 对任务  $a_i \in A_j$ , 记其被分配的 SM 资源比例为  $s_i$ , 并满足  $0 \leq s_i \leq 1$ 。由于同一节点上的任务共享 GPU 计算资源, 因此 SM 比例变量需满足总资源约束:

$$\sum_{a_i \in A_j} s_i \leq 1 \quad (30)$$

对于任务  $a_i$ ，在给定资源比例  $s_i$  后，其执行时延由前述时延拟合模型给出：

$$\hat{T}_i^{exe} = G_{\theta^*}(F_{m,i}, F_{w,i}, s_i) \quad (31)$$

得到节点  $q_j$  的资源分配比例组合  $S_j = \{s_i \mid a_i \in A_j\}$ ，因此第二阶段细粒度资源分配问题的优化目标可定义为：在满足总资源约束的条件下，使节点内任务在当前 SM 资源分配比例设置下的总体预测时延最小。其目标函数可表示为：

$$\min_{S_j} J_{lat}^{(j)} = \sum_{a_i \in A_j} \omega_i \hat{T}_i^{exe} = \sum_{a_i \in A_j} \omega_i G_{\theta^*}(F_{m,i}, F_{w,i}, s_i) \quad (32)$$

其中， $\omega_i$  表示任务  $a_i$  的权重，用于刻画不同并行形态的分布式推理任务优先级、服务等级或调度策略中的重要性。由此，第二阶段调度借助神经网络预测不同资源配置下的任务执行时延，并以此实现面向任务特征的细粒度 GPU 资源调控，从而缓解多任务共驻带来的资源竞争问题，提升节点内部的执行效率。

### 4.3 系统设计与实现

本研究的目标是面向边缘场景下分布式 AI 推理任务的部署与执行优化问题，构建一套基于 k3s 容器集群的分布式推理调度系统，用于实现通信感知的任务放置与细粒度 GPU 资源分配。如图所示，系统整体由四个模块组成：1) 任务输入与特征构建模块；2) 通信模式抽取与代价预测模块；3) 通信感知与细粒度资源协同调度模块；4) 运行监测与反馈更新模块。系统的核心思想是：将分布式推理任务在不同并行机制下的通信行为与资源需求进行结构化建模，在集群级以通信时延预测结果为依据完成任务放置，在节点级进一步结合多任务共驻条件下的资源竞争特征进行细粒度 GPU 资源调度，从而实现通信优化与资源优化的协同，提升边缘容器集群环境下分布式推理系统的整体执行效率与可扩展性。概览图如图 5 所示：

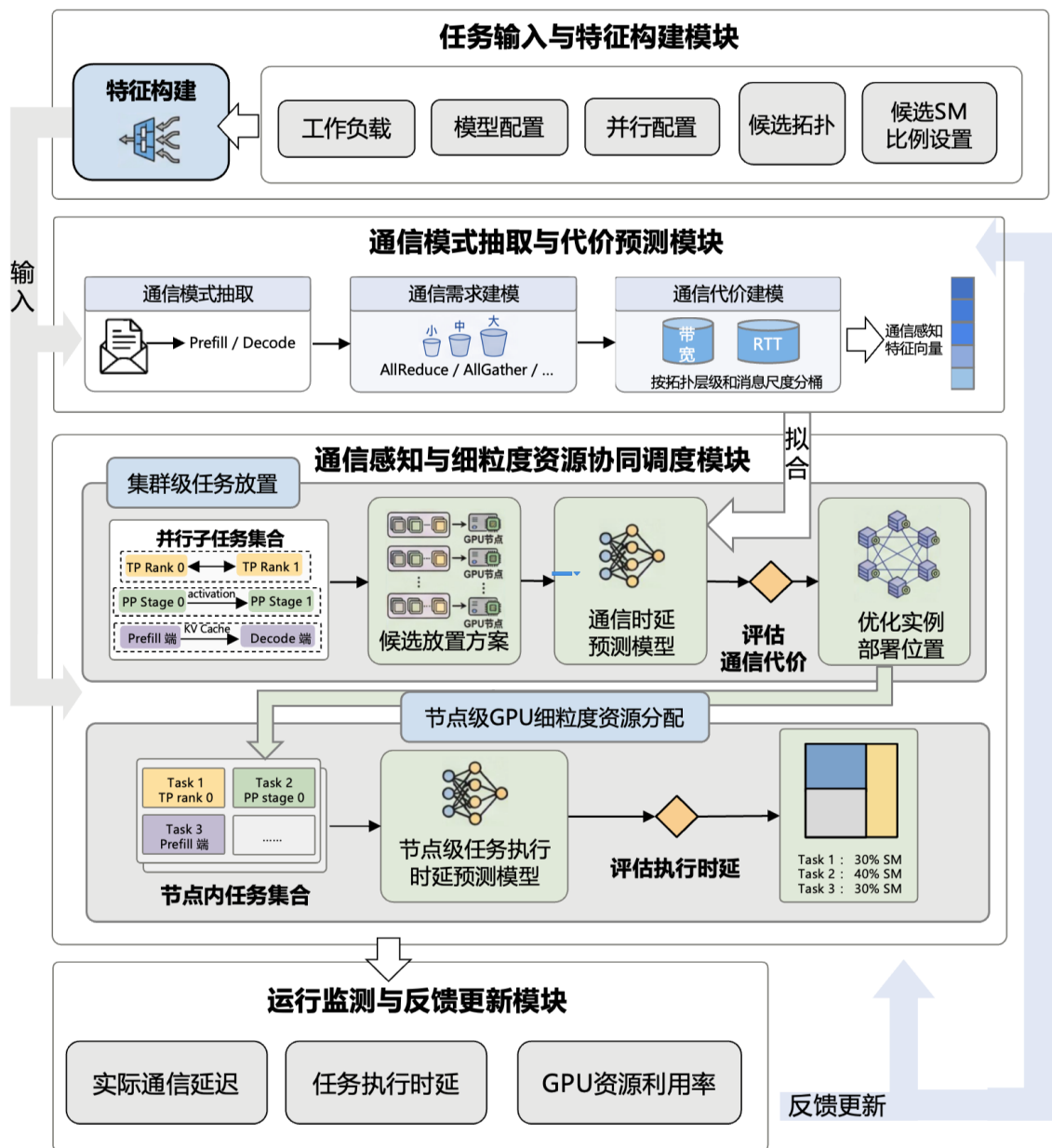


图5 边缘场景下基于 k3s 的分布式推理调度系统

### (1) 任务输入与特征构建模块

任务输入与特征构建模块负责收集并组织分布式推理调度所需的各类输入信息，为后续通信代价评估和调度决策提供统一的特征表示。该模块首先接收来自推理服务框架的推理任务，包括模型类型、输入 prompt 长度上限、输出 token 上限以及并行配置等基础信息。同时，从集群侧获取候选部署节点、节点资源状态、节点间拓扑关系以及同节点/跨节点、同机架/跨机架等拓扑层级特征，用于后续候选拓扑方案构造。在此基础上，该模块利用通信模式抽取与代价预测方法，对任务工作负载、模型配置和候选拓扑信息进行组织，形成面向通信时延拟合模型的输入表示。对于节点级资源调度，该模块构造任务类型特征、工作负载特征和 MPS/SM 资源比例特征，为节点级执行时延拟合模型提供输入。由此，任务输入与特征构

建模块作为整个系统的数据入口和建模前端，起到统一抽象输入信息并组织调度可用特征的作用。

### **(2) 通信模式抽取与代价预测模块**

通信模式抽取与代价预测模块负责在调度前对分布式推理任务的通信行为进行建模与刻画，是连接任务特征构建与调度决策的关键环节。该模块首先从任务输入与特征构建模块获取工作负载、模型配置及并行策略等信息，将分布式推理过程划分为 **Prefill** 与 **Decode** 两个阶段，并在此基础上对推理执行过程中产生的通信事件进行抽取与解析。针对不同通信原语以及不同并行规模，该模块进一步分析通信事件在通信规模与通信频率上的差异特征，为刻画通信模式的结构化特征，模块在通信需求建模阶段引入消息尺度分桶机制，将通信事件划分为小、中、大不同粒度区间，并在事件级别构建统一的通信表征形式。同时，为消除不同通信原语与并行规模带来的差异，模块引入等效字节需求与等效轮次需求等统一折算指标，对通信事件进行标准化描述，从而在统一尺度下反映不同通信行为对系统性能的影响。在此基础上，模块进一步在推理阶段和消息尺度两个维度上对通信事件进行聚合，形成结构化的通信需求向量。在通信代价建模阶段，模块进一步结合候选部署拓扑信息，对不同消息尺度在不同拓扑层级下的通信开销进行建模，刻画有效带宽与通信延迟等特征，并构建通信时延预测模型，实现对工作负载——模型配置——部署拓扑组合下通信开销的准确估计。

### **(3) 调度决策与执行模块**

调度决策与执行模块在逻辑上分为两个相互衔接的部分，即集群级任务放置子模块与节点级 GPU 资源分配子模块。集群级任务放置子模块对应第一阶段调度，以分布式推理实例为调度对象，首先根据实例内部并行子任务的组成结构生成若干候选放置方案，并据此形成实例级候选拓扑组。其次，调用训练得到的通信时延拟合模型，对每个实例在不同候选放置方案下的整体通信代价进行评估。在满足唯一放置约束与节点资源容量约束的前提下，子模块选择预测通信时延低的候选方案，从而确定实例内部各并行子任务在集群范围内的放置位置。该过程的目标是尽可能降低跨节点与跨机架通信开销，提高实例执行的整体通信效率。节点级 GPU 资源分配子模块对应第二阶段调度。在第一阶段给定实例放置结果后，该子模块进一步聚合同一 GPU 节点上的共驻任务，形成节点内任务集合，以任务类型特征、工作负载特征和 MPS/SM 资源比例设置为输入，调用节点级任务执行时延拟合模型，评估不同资源配置下的任务执行性能。在满足节点内总 SM 资源约束的条件下，子模块搜索更优的资源比例组合，使节点内任务的总体预测执行时延最小。由此，调度决策与执行模块实现了分布式推理任务在容器集群中的通信优化和资源优化的协同。

#### (4) 运行监测与反馈更新模块

运行监测与反馈更新模块负责对系统运行过程中的任务执行状态、通信表现和资源利用情况进行持续监测，并为系统后续优化提供反馈支撑。具体而言，该模块跟踪推理实例在不同放置方案下的实际通信延迟、节点内任务在不同资源配置下的实际执行时延、GPU 资源利用率以及共驻任务间的资源竞争情况，并将这些运行数据回传至系统侧，用于评估当前调度策略的有效性。对于集群级任务放置过程，该模块关注实例在不同部署位置下的真实通信代价表现，以检验通信时延拟合模型在实际运行场景中的预测效果；对于节点级资源分配过程，该模块关注不同 MPS/SM 资源比例配置下的任务执行延迟与节点资源利用率，以评估节点级资源调度策略的优化效果。基于这些运行反馈，系统逐步修正候选拓扑的代价建模、更新通信时延拟合模型和节点级执行时延预测模型，并进一步调整调度参数，从而形成“输入构建—调度决策—运行反馈校正”的闭环机制。

#### 4.4 可行性分析

本研究掌握了相关理论知识和前沿状态，所设计的面向 AI 推理任务的通信模式抽取与代价预测方法能够拟合通信时延预测模型，并基于不同模型配置、工作负载、并行配制、拓扑链路对通信时延进行预测，为后续任务部署优化提供支撑。

##### (1) 技术实现可行性

本研究的整体技术链条清晰，且各部分均具备较好的实现基础。首先，在通信建模方面，研究围绕分布式 AI 推理任务在不同并行机制下的通信行为展开分析，将推理过程按 **Prefill** 与 **Decode** 两阶段进行划分，并结合消息尺度区间、候选拓扑特征和工作负载特征构建通信时延拟合模型。这一建模过程建立在容器集群、GPU 节点通信特征以及深度神经网络回归模型的基础之上，所需的通信采集、模型训练与参数拟合方法均已有较成熟的实验条件和实现路径。其次，在调度机制方面，本研究提出的面向边缘集群的通信感知与细粒度资源协同调度机制分别对应集群级任务放置与节点级 GPU 细粒度资源分配，前者以通信时延拟合结果为依据进行候选方案评估，后者以节点级执行时延预测模型为支撑进行资源比例优化，其方法形式与系统实现路径均较为明确。最后，在系统实现层面，本研究采用 k3s 作为边缘场景下的轻量级容器集群平台，结合 GPU 节点管理、任务部署、资源控制和运行监测模块构建分布式推理调度系统。k3s 本身具备轻量部署、易扩展、易管理等特点，适合边缘环境中的实验验证。因此，从模型构建、调度设计到系统落地，本研究拟开展的研究工作在技术上具有较高的可实现性。

##### (2) 经济实现可行性

本研究所需的实验环境主要包括 GPU 服务器、边缘节点、k3s 容器集群环境以及分布式推理运行框架等。这些软硬件基础大多可以依托实验室现有的 GPU 设备、服务器资源及开源软件平台完成部署，不需要额外投入大规模专用设备成本。与此同时，本研究拟采用的核心系统组件，如 k3s、容器运行时、分布式推理框架以及模型训练与调度实现所依赖的软件工具，均具备良好的开源基础和可获得性，因此在系统搭建和实验实施过程中不会产生过高的软件采购成本。总体而言，本研究在实验平台、开发环境和系统实现工具方面均具有较好的资源基础，经济投入可控，具备良好的经济实现可行性。

### （3）应用实现可行性

本研究面向的是边缘场景下分布式 AI 推理任务的部署与执行优化问题，其应用背景明确，研究结果具有较强的实际落地价值。随着边缘侧大模型推理需求的增长，如何在资源受限、拓扑异构的容器集群环境中实现高效部署和稳定运行，已成为具有现实意义的关键问题。本研究提出的两阶段调度系统以通信感知和资源感知为核心，通过集群级降低跨节点通信代价、节点级缓解 GPU 资源竞争，能够较好契合边缘推理场景中对低时延、高资源利用率和弹性部署的实际需求。同时，本研究采用的系统平台和实现框架均基于通用的容器编排与 GPU 资源管理机制，具有较好的可迁移性和可扩展性，便于在不同规模的边缘集群中进行扩展验证。因此，从应用场景适配性和系统推广可行性来看，本研究具有较高的应用实现可行性。

### （4）规范性与研究过程可控性

本研究拟采用的系统平台和开发环境主要基于开源框架与自主实现算法，不涉及受限商业接口或高风险数据来源，也不涉及社会伦理风险较高的应用场景。相关模型设计与系统实现均基于现有公开研究工作进行分析、改进与扩展，研究过程具备较强的规范性与可控性。由此，本研究在整体推进过程中不存在明显的伦理或合规障碍，具备稳定开展的条件。

## 五、参考文献

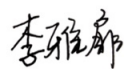
- [1] 算网融合产业及标准推进委员会. “边缘计算+”技术白皮书（2022 年）[J]. 北京, 2022.
- [2] 英特尔公司. 边缘 AI 驱动，助力新质生产力——英特尔工业人工智能白皮书（2025 年版）[J]. 北京: 英特尔公司, 2025.
- [3] 贾庆民, 丁瑞, 刘辉, 等. 算力网络研究进展综述[J]. Chinese Journal of Network & Information Security, 2021, 7(5).
- [4] Tang Z, Mou F, Lou J, et al. Joint resource overbooking and container scheduling in edge computing[J]. IEEE Transactions on Mobile Computing, 2024, 23(12): 10903-10917.

- [5] Mao Y, Oak J, Pompili A, et al. Draps: Dynamic and resource-aware placement scheme for docker containers in a heterogeneous cluster[C]//2017 IEEE 36th International Performance Computing and Communications Conference (IPCCC). IEEE, 2017: 1-8.
- [6] Gu R, Chen Y, Liu S, et al. Liquid: Intelligent resource estimation and network-efficient scheduling for deep learning jobs on distributed GPU clusters[J]. IEEE Transactions on Parallel and Distributed Systems, 2021, 33(11): 2808-2820.
- [7] Xia Z, Hao Y, Duan J, et al. Towards optimal preemptive gpu time-sharing for edge model serving[C]//Proceedings of the 9th International Workshop on Container Technologies and Container Clouds. 2023: 13-18.
- [8] Fiori S, Abeni L, Cucinotta T. Rt-kubernetes: containerized real-time cloud computing[C]//Proceedings of the 37th ACM/SIGAPP Symposium on Applied Computing. 2022: 36-39.
- [9] Ali B, Golec M, Gill S S, et al. EdgeAIBus: AI-driven joint container management and model selection framework for heterogeneous edge computing[J]. IEEE Transactions on Parallel and Distributed Systems, 2025.
- [10] Kwon W, Li Z, Zhuang S, et al. Efficient memory management for large language model serving with pagedattention[C]//Proceedings of the 29th symposium on operating systems principles. 2023: 611-626.
- [11] Leviathan Y, Kalman M, Matias Y. Fast inference from transformers via speculative decoding[C]//International Conference on Machine Learning. PMLR, 2023: 19274-19286.
- [12] Cai T, Li Y, Geng Z, et al. MEDUSA: Simple LLM inference acceleration framework with multiple decoding heads[C]//Proceedings of the 41st International Conference on Machine Learning. 2024: 5209-5235.
- [13] Lin J, Tang J, Tang H, et al. Awq: Activation-aware weight quantization for on-device llm compression and accelerate[J]. Proceedings of machine learning and systems, 2024, 6: 87-100.
- [14] Song Y, Mi Z, Xie H, et al. Powerinfer: Fast large language model serving with a consumer-grade gpu[C]//Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles. 2024: 590-606.
- [15] Dao T. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning[C]//The Twelfth International Conference on Learning Representations.
- [16] Yu G I, Jeong J S, Kim G W, et al. Orca: A distributed serving system for {Transformer-Based} generative models[C]//16th USENIX symposium on operating systems design and implementation (OSDI 22). 2022: 521-538.
- [17] Zheng L, Yin L, Xie Z, et al. Sglang: Efficient execution of structured language model programs[J]. Advances in neural information processing systems, 2024, 37: 62557-62583.
- [18] Sheng Y, Zheng L, Yuan B, et al. Flexgen: High-throughput generative inference of large language models with a single gpu[C]//International Conference on Machine Learning. PMLR, 2023: 31094-31116.



- [19] Wan Z, Wang X, Liu C, et al. Efficient large language models: A survey[J]. arXiv preprint arXiv:2312.03863, 2023.
- [20] Huang J, Chang K C C. Towards reasoning in large language models: A survey[C]//Findings of the association for computational linguistics: ACL 2023. 2023: 1049-1065.
- [21] Li Z Z, Zhang D, Zhang M L, et al. From system 1 to system 2: A survey of reasoning large language models[J]. arXiv preprint arXiv:2502.17419, 2025.
- [22] Weng Q, Xiao W, Yu Y, et al. {MLaaS} in the wild: Workload analysis and scheduling in {Large-Scale} heterogeneous {GPU} clusters[C]//19th USENIX symposium on networked systems design and implementation (NSDI 22). 2022: 945-960.
- [23] Hindman B, Konwinski A, Zaharia M, et al. Mesos: A platform for {Fine-Grained} resource sharing in the data center[C]//8th USENIX Symposium on Networked Systems Design and Implementation (NSDI 11). 2011.
- [24] Mohan J, Phanishayee A, Kulkarni J, et al. Synergy: Resource sensitive DNN scheduling in multi-tenant clusters[J]. arXiv preprint arXiv:2110.06073, 2021.
- [25] Senjab K, Abbas S, Ahmed N, et al. A survey of Kubernetes scheduling algorithms[J]. Journal of Cloud Computing, 2023, 12(1): 87.
- [26] Hsieh T T, Lee C R. Voda: A gpu scheduling platform for elastic deep learning in kubernetes clusters[C]//2023 IEEE International Conference on Cloud Engineering (IC2E). IEEE, 2023: 131-140.
- [27] Rac S, Brorsson M. Cost-effective scheduling for kubernetes in the edge-to-cloud continuum[C]//2023 IEEE International Conference on Cloud Engineering (IC2E). IEEE, 2023: 153-160.
- [28] Ghodsi A, Zaharia M, Hindman B, et al. Dominant resource fairness: Fair allocation of multiple resource types[C]//8th USENIX symposium on networked systems design and implementation (NSDI 11). 2011.
- [29] Grandl R, Ananthanarayanan G, Kandula S, et al. Multi-resource packing for cluster schedulers[J]. ACM SIGCOMM Computer Communication Review, 2014, 44(4): 455-466.
- [30] Gu J, Chowdhury M, Shin K G, et al. Tiresias: A {GPU} cluster manager for distributed deep learning[C]//16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19). 2019: 485-500.
- [31] Xiao W, Bhardwaj R, Ramjee R, et al. Gandiva: Introspective cluster scheduling for deep learning[C]//13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18). 2018: 595-610.
- [32] Han Z, Zhou R, Xu C, et al. Inss: An intelligent scheduling orchestrator for multi-gpu inference with spatio-temporal sharing[J]. IEEE Transactions on Parallel and Distributed Systems, 2024, 35(10): 1735-1748.
- [33] Chen Z, Zhao X, Zhi C, et al. Deepboot: Dynamic scheduling system for training and inference deep learning tasks in gpu cluster[J]. IEEE transactions on parallel and distributed systems, 2023, 34(9): 2553-2567.
- [34] Zeng L, Zhang G, Peng J, et al. Kant: An Efficient Unified Scheduling System for Large-Scale AI Clusters[J]. arXiv preprint arXiv:2510.01256, 2025.

- [35] Struhár V, Craciunas S S, Ashjaei M, et al. Hierarchical resource orchestration framework for real-time containers[J]. ACM Transactions on Embedded Computing Systems, 2024, 23(1): 1-24.
- [36] Wu B, Zhang Z, Bai Z, et al. Transparent {GPU} sharing in container clouds for deep learning workloads[C]//20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23). 2023: 69-85.

研究生签名 

年 月 日

(注：本页不够可附页)

## 二、学位论文工作实施计划

### (一) 论文的理论分析与硬件要求及其预期达到的水平与结果

#### 理论成果

- (1) 面向分布式推理任务的通信模式抽取与代价预测方法
- (2) 面向边缘集群的通信感知与细粒度资源协同调度机制

#### 系统成果

本硕士论文将根据 1) 面向分布式推理任务的通信模式抽取与代价预测方法, 2) 面向边缘集群的通信感知与细粒度资源协同调度机制, 设计并实现边缘场景下基于 k3s 的分布式推理调度系统, 提升边缘容器集群中分布式推理系统的整体执行效率。

#### 成果形式

- (1) 在国内外核心期刊或学术会议上发表学术论文 1-2 篇。
- (2) 完成东南大学软件学院硕士论文 1 篇。
- (3) 边缘场景下基于 k3s 的分布式推理调度系统。

### (二) 论文工作进度与安排

| 起讫日期             | 工 作 内 容 和 要 求      | 备 注 |
|------------------|--------------------|-----|
| 2025 年 09 月-12 月 | 进行相关理论学习;          |     |
| 2026 年 01 月-05 月 | 阅读文献, 完成理论分析和方案设计; |     |

|                          |                                                                                                                           |  |
|--------------------------|---------------------------------------------------------------------------------------------------------------------------|--|
| 2026 年 06 月-12 月         | 完善理论方案，进行系统开发与测试；                                                                                                         |  |
| 2027 年 01 月-04 月         | 毕业论文撰写；                                                                                                                   |  |
| 2027 年 04 月-06 月         | 准备毕业答辩。                                                                                                                   |  |
|                          |                                                                                                                           |  |
|                          |                                                                                                                           |  |
|                          |                                                                                                                           |  |
| 学校指导教师对开题<br>报告的综合意见     | <p>该开题报告内容完备，规划合理，通过</p> <p style="text-align: right;">指导教师（签字）</p> <p style="text-align: right;">年    月    日</p>         |  |
| 校外指导<br>教师对开题报告的综<br>合意见 | <p>该开题报告内容完备，规划合理，通过</p> <p style="text-align: right;">指导教师（签字）<br/>宋明哲</p> <p style="text-align: right;">年    月    日</p> |  |